

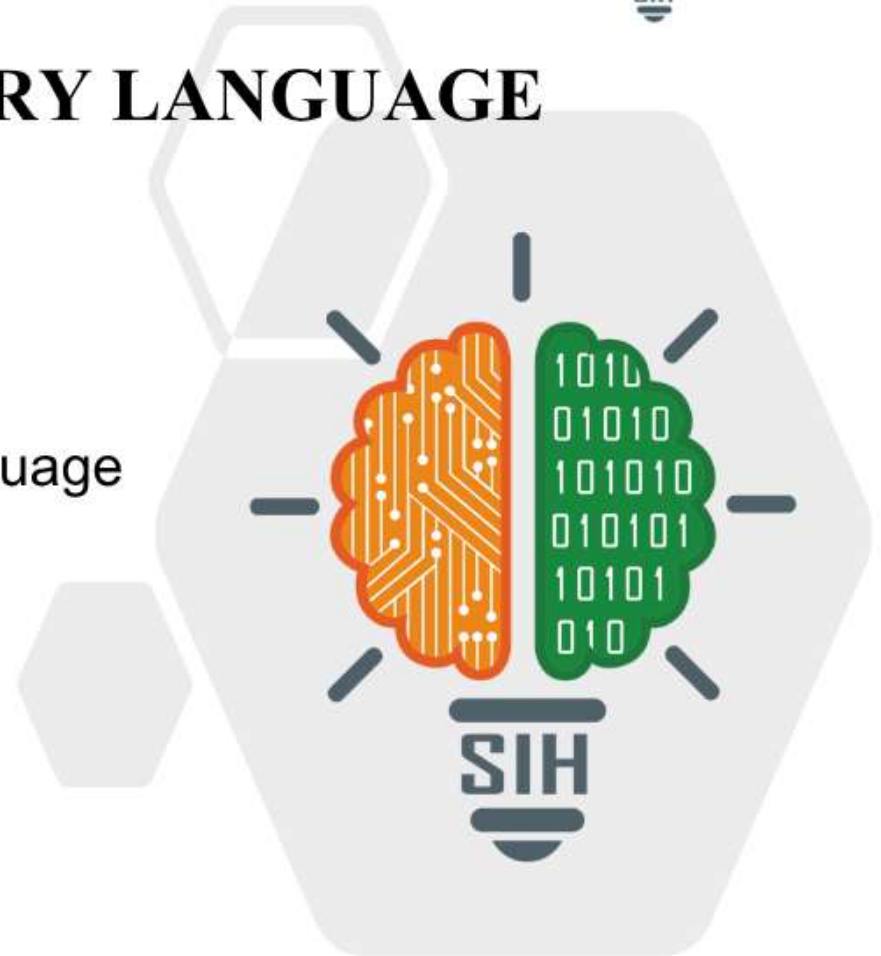
SMART INDIA HACKATHON 2026



SMART INDIA
HACKATHON
2026

MINI FORENSIC QUERY LANGUAGE

- **Problem Statement ID** – SIH26148
- **Problem Statement Title**- Creation of scripts/functions with new programming language to commence Computer & Network forensic analysis without triggering security solutions
- **Theme**- Cyber Security & Blockchain
- **PS Category**- Software
- **Team ID**- 31
- **Team Name** : GVAIET Algorithm Avenger





IDEA TITLE



❖ Proposed Solution :-

- We built a compact DSL (fixed vocabulary + regex parser) that converts plain-English investigator queries like “show all failed logins after 2am” into a structured Query object, then an interpreter applies that Query as filters over a parsed log file to return matching rows or counts instantly.
- Investigators currently sift through raw logs manually or write ad-hoc scripts/SQL; our DSL lets a non- programmer type a natural investigative question and get precise, filtered results in seconds, no query language training required.
- The parser and interpreter are decoupled stages (text to Query struct, Query struct to filtered results), so the same architecture can later swap in a formal grammar or LLM-based parsing without a redesign, unlike typical single-pass keyword search tools.



TECHNICAL APPROACH



- Python 3 (standard library only) for the parser and interpreter; regex-based tokenizer for the DSL grammar; CSV for log ingestion; optional Flask web UI for the query console. No external services or paid APIs required for the MVP.
- Pipeline: Query text (plain English) -> Parser (matches fixed vocabulary/regex rules -> structured Query object) -> Interpreter (applies Query as filters over parsed log rows) -> Results table or count. Built and demoed end-to-end as a working CLI prototype within a 3-hour window.



FEASIBILITY AND VIABILITY



- Feasible as an MVP with zero external dependencies: built and demoed a working end-to-end prototype (parser + interpreter + sample log query) inside a single 3-hour hackathon window using only Python's standard library.
- Fixed-vocabulary parser may miss free-form phrasing; large log files need indexing for real-time speed; ambiguous queries risk silent misinterpretation.
- Mitigation: expand vocabulary/fuzzy matching iteratively, add log indexing for scale, and return a clarifying prompt whenever a query can't be parsed confidently instead of guessing.



IMPACT AND BENEFITS



- Lowers the technical barrier for investigators, SOC analysts, auditors, and compliance teams to query security/system logs directly, without waiting on a data or engineering team for every ad-hoc question.
- Faster incident response and investigation turnaround, reduced dependency on specialist query-writers, and a reusable architecture that extends to any structured log source (auth systems, firewalls, application logs).



RESEARCH AND REFERENCES



- Martin Fowler, "Domain-Specific Languages" (2010) — DSL design principles; Python re (regex) module documentation, docs.python.org; Splunk SPL and Kusto Query Language (KQL) as prior art for investigator-facing log query languages; OWASP Logging Cheat Sheet for log field conventions.